

COMPUTER PROGRAMMING LAB REPORT

Lab Report 13

Title: Structures in C++

1. OBJECTIVE

- To understand the concept of structures in C++
- To learn how to create and use user-defined data types
- To store multiple related values under one structure
- To practice accessing structure members using dot operator

2. THEORY

A **structure (struct)** in C++ is a user-defined data type that allows grouping different types of variables under a single name. Unlike arrays, structures can store multiple data types such as int, string, float, and char.

Structures are useful when we want to represent real-world objects like:

- Student records
- Book information
- Person details

Each variable inside a structure is called a **member**, and it is accessed using the **dot (.) operator**.

3. SOFTWARE USED

- DEV-C++ IDE
- GNU C++ Compiler

4. PROGRAMS

Program 1 — Person Structure

```
#include <iostream>
using namespace std;
struct Person {           // structure to store personal information
    string name;           // stores person name
    int age;               // stores person age
    string address;        // stores person address
};
```

```

};
int main() {                                // main function starts execution
    Person person1 = {"Danyal Hassan", 19, "Karak, KPK"};
    cout << "Name: " << person1.name << endl;    // displaying name
    cout << "Age: " << person1.age << endl;      // displaying age
    cout << "Address: " << person1.address << endl;
    // displaying address
    return 0;                                // program ends successfully
}

```

Output

Name: Danyal Hassan
 Age: 19
 Address: Karak, KPK

Program 2 — Student Structure

```

#include <iostream>
using namespace std;
struct Student {
    string name;           // stores student name
    int age;               // stores student age
    char grade;            // stores student grade
};
int main() {              // main function starts
    Student student1 = {"Danyal Hassan", 19, 'A'};
    // initializing student data
    cout << "Name: " << student1.name << endl;    // display name
    cout << "Age: " << student1.age << endl;      // display age
    cout << "Grade: " << student1.grade << endl;  // display grade
    return 0;              // program ends
}

```

Output

Name: Danyal Hassan
 Age: 19
 Grade: A

Program 3 — Book Structure (Multiple Objects)

```
#include <iostream>
using namespace std;
struct Book {
    string title;           // stores book title
    string author;          // stores author name
    float price;            // stores book price
    int pages;              // stores number of pages
};
int main() {               // main function starts
    Book book1 = {"C++ Basics", "Bjarne Stroustrup", 500.0, 300};
    // first book data
    Book book2 = {"Engineering Mechanics", "Meriam", 750.5, 650};
    // second book data
    cout << "Book 1 Details:" << endl;    // heading for first book
    cout << "Title: " << book1.title << endl; // display title
    cout << "Author: " << book1.author << endl; // display author
    cout << "Price: " << book1.price << endl; // display price
    cout << "Pages: " << book1.pages << endl; // display pages
    cout << endl;                          // blank line for separation
    cout << "Book 2 Details:" << endl;    // heading for second book
    cout << "Title: " << book2.title << endl; // display title
        cout << "Author: " << book2.author << endl; // display author
        cout << "Price: " << book2.price << endl; // display price
        cout << "Pages: " << book2.pages << endl; // display pages
    return 0;                // program ends successfully
}
```

Output

Book 1 Details:
Title: C++ Basics
Author: Bjarne Stroustrup
Price: 500
Pages: 300

Book 2 Details:
Title: Engineering Mechanics
Author: Meriam
Price: 750.5
Pages: 650

5. CONCLUSION

In this lab, we learned how to use **structures in C++** to store and manage related data efficiently. We created different structure types such as Person, Student, and Book, and accessed their members using the dot operator. This lab helped improve understanding of user-defined data types and data organization in C++ programming.